FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Julius Jojo C. Sabulao Jr
Section:	AN43
Professor:	Crisola Tan
Date:	10/11/2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩

```
import random
from pathlib import Path

import agentpy as ap
import matplotlib.pyplot as plt


class RandomWalker(ap.Agent):
    """Agent that randomly moves up, down, left, or right."""

    def setup(self):
        width, height = self.model.p.grid_size
        self.position = (
            self.model.rng.randrange(width),
            self.model.rng.randrange(height),
        )

    def move(self):
        x, y = self.position
        width, height = self.model.p.grid_size
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        self.model.rng.shuffle(directions)

        possible_positions = []
        for dx, dy in directions:
            # Restrict every coordinate to the grid boundaries.
            new_x = max(0, min(width - 1, x + dx))
            new_y = max(0, min(height - 1, y + dy))
            candidate = (new_x, new_y)
            if candidate not in possible_positions:
                possible_positions.append(candidate)
```

```
        # Modified rule: select an unoccupied cell when one is available.
        if self.model.p.avoid_collisions:
            occupied = {
                other.position
                for other in self.model.agents
                if other is not self
            }
            free_positions = [
                position
                for position in possible_positions
                if position not in occupied
            ]
            if free_positions:
                possible_positions = free_positions

        self.position = self.model.rng.choice(possible_positions)


class RandomWalkModel(ap.Model):
    """Multi-agent random walk model with path tracking."""

    def setup(self):
        self.rng = random.Random(self.p.seed)
        self.agents = ap.AgentList(self, self.p.num_agents, RandomWalker)
        self.agents.setup()
        self.paths = {
            agent.id: [agent.position]
            for agent in self.agents
        }

    def step(self):
        for agent in self.agents:
            agent.move()
            self.paths[agent.id].append(agent.position)

    def run_simulation(self):
        for _ in range(self.p.num_steps):
            self.step()
        return self


def run_scenario(num_agents, grid_size, num_steps, seed, avoid_collisions):
    parameters = {
        "num_agents": num_agents,
```

```
        "grid_size": (grid_size, grid_size),
        "num_steps": num_steps,
        "seed": seed,
        "avoid_collisions": avoid_collisions,
    }
    model = RandomWalkModel(parameters)
    model.setup()
    return model.run_simulation()


def draw_paths(model, axis, title):
    """Draw the complete path of every agent."""
    width, height = model.p.grid_size

    for number, agent in enumerate(model.agents, start=1):
        path = model.paths[agent.id]
        x_values, y_values = zip(*path)
        axis.plot(
            x_values,
            y_values,
            marker=".",
            linewidth=1.2,
            label=f"Agent {number}",
        )
        axis.scatter(
            x_values[0],
            y_values[0],
            marker="o",
            s=45,
            edgecolor="black",
        )
        axis.scatter(
            x_values[-1],
            y_values[-1],
            marker="X",
            s=70,
            edgecolor="black",
        )

    axis.set_xlim(-0.5, width - 0.5)
    axis.set_ylim(-0.5, height - 0.5)
    axis.set_xticks(range(width))
    axis.set_yticks(range(height))
    axis.set_xlabel("X position")
```

```
    axis.set_ylabel("Y position")
    axis.set_title(title)
    axis.grid(True, alpha=0.35)
    axis.legend(fontsize=8, loc="upper left", bbox_to_anchor=(1.02, 1))


def save_final_positions(model, filename):
    """Print and save final positions for laboratory task 3."""
    lines = ["FINAL AGENT POSITIONS"]

    for number, agent in enumerate(model.agents, start=1):
        line = f"Agent {number}: {agent.position}"
        lines.append(line)
        print(line)

    Path(filename).write_text("\n".join(lines), encoding="utf-8")


def main():
    print("RANDOM WALK SIMULATION USING AGENTPY")
    num_agents = int(input("Enter number of agents: "))
    grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
    num_steps = int(input("Enter number of steps: "))

    choice = input("Use collision avoidance? (y/n): ").strip().lower()
    avoid_collisions = choice == "y"

    # Scenario 1 uses the values entered by the user.
    scenario_1 = run_scenario(
        num_agents,
        grid_size,
        num_steps,
        seed=42,
        avoid_collisions=avoid_collisions,
    )

    print("\nScenario 1:")
    save_final_positions(scenario_1, "TA1_final_positions.txt")

    # Scenario 2 has twice as many agents and steps for comparison.
    scenario_2 = run_scenario(
        max(2, num_agents * 2),
        grid_size,
        max(4, num_steps * 2),
```

```
        seed=99,
        avoid_collisions=avoid_collisions,
    )

    # Save a full path image for Scenario 1.
    figure_1, axis_1 = plt.subplots(figsize=(8, 7))
    draw_paths(scenario_1, axis_1, "Scenario 1 - Agent Paths")
    figure_1.tight_layout()
    figure_1.savefig("TA1_agent_paths.png", dpi=200, bbox_inches="tight")

    # Save the side-by-side comparison required by laboratory task 4.
    figure_2, axes = plt.subplots(1, 2, figsize=(14, 6))
    draw_paths(scenario_1, axes[0], "Scenario 1 - Original Parameters")
    draw_paths(scenario_2, axes[1], "Scenario 2 - More Agents and Steps")
    figure_2.tight_layout()
    figure_2.savefig("TA1_scenario_comparison.png", dpi=200, bbox_inches="tight")

    print("\nFiles created:")
    print("- TA1_final_positions.txt")
    print("- TA1_agent_paths.png")
    print("- TA1_scenario_comparison.png")
    print("Close the graph window to finish the program.")

    plt.show()


if __name__ == "__main__":
    main()
```

Step 5: Answer the Questions

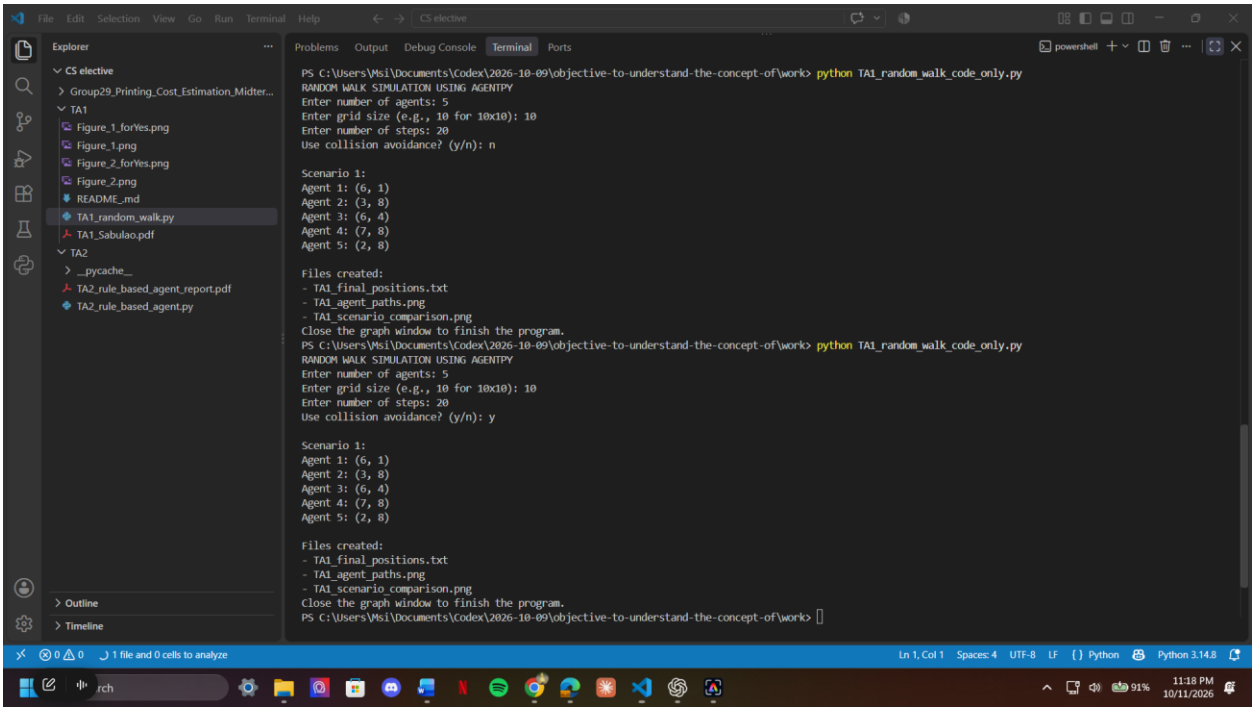
1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

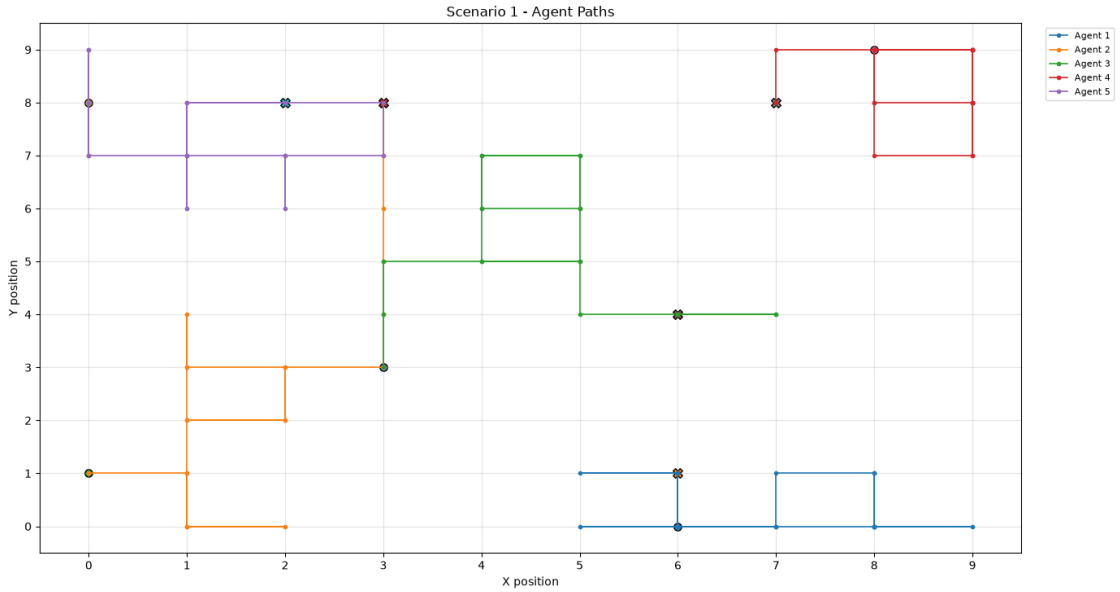
1. When the number of agents is high compared to the grid size, the environment becomes crowded. Agents are more likely to occupy the same cells, cross paths, or form clusters. This can be observed in Figure 2, where the scenario with ten agents has denser and more overlapping paths than the scenario with five agents.
2. Increasing the number of steps gives the agents more opportunities to move and explore the grid. Their paths become longer and generally cover a larger portion of the

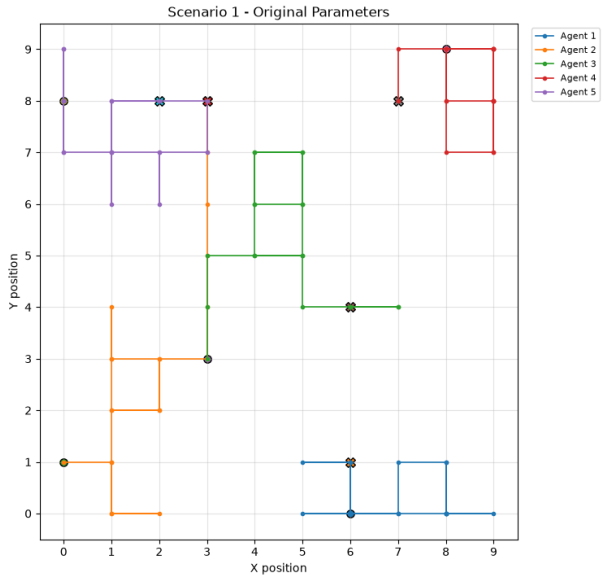
environment. However, because movement is random, some agents may repeatedly visit the same cells instead of continuously moving farther away.

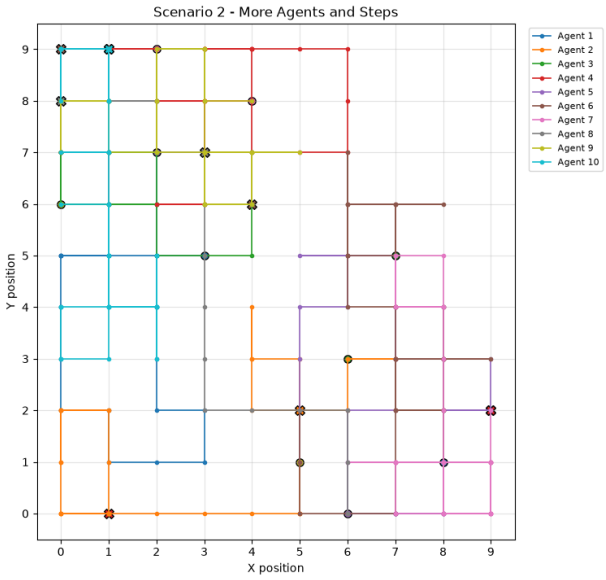
3. No, the agents do not move outside the grid. The program restricts every new X and Y coordinate between 0 and grid_size - 1. For a 10×10 grid, the valid coordinates are from 0 to 9. This boundary rule keeps all agents inside the environment.
4. The simulation can represent real-life processes involving random movement. For example, agents can represent particles spreading during diffusion, animals searching for food, people moving through a crowd, or robots exploring an area. The movement rules and environmental conditions can be modified to create more realistic behavior.



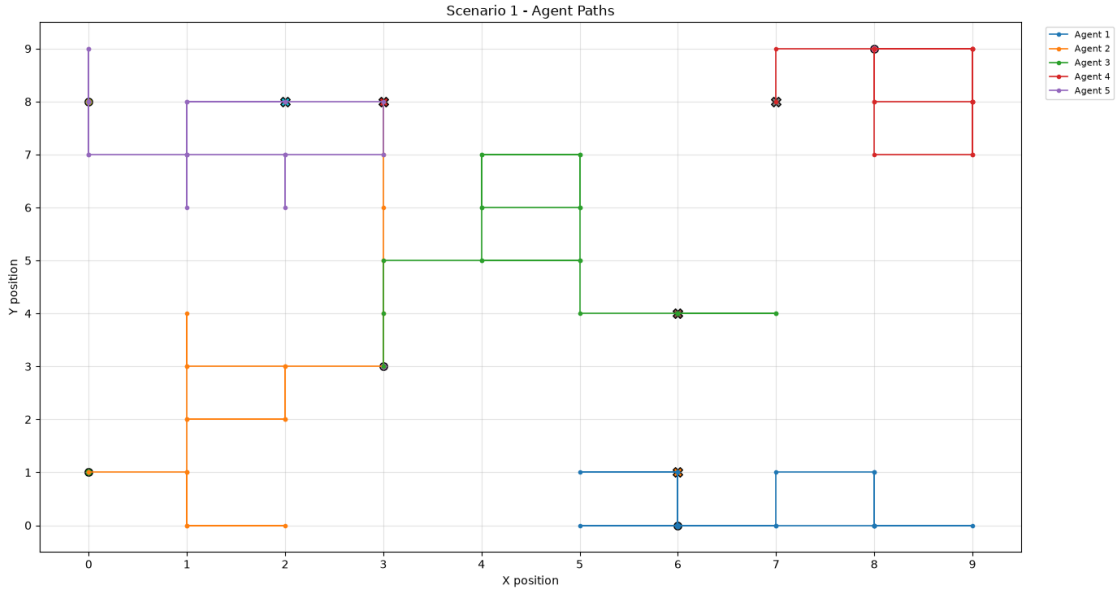
OFF

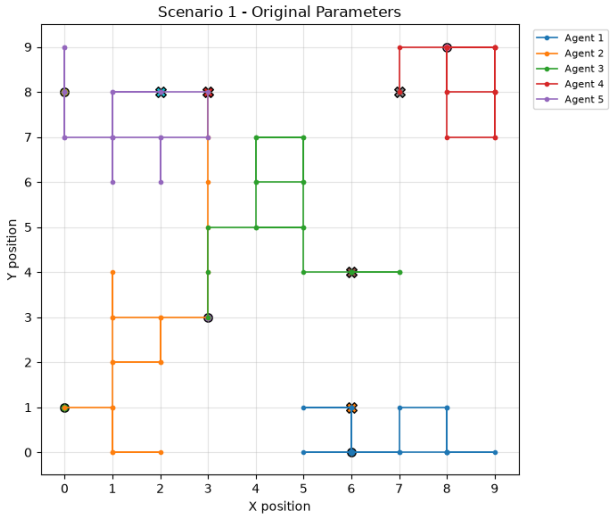

Scenario 1 - Agent Paths
Agent 1
Agent 2
Agent 3
Agent 4
Agent 5
Y position
X position


Scenario 1 - Original Parameters
Agent 1
Agent 2
Agent 3
Agent 4
Agent 5
Y position
X position


Scenario 2 - More Agents and Steps
Agent 1
Agent 2
Agent 3
Agent 4
Agent 5
Agent 6
Agent 7
Agent 8
Agent 9
Agent 10
Y position
X position

ON







Overall Observation and Comparison: Collision Avoidance OFF vs. ON

Two simulations were conducted using the same basic parameters. In the first simulation, collision avoidance was set to No, allowing agents to move without checking whether another agent occupied the destination cell. This resulted in more overlapping positions and intersecting paths. In the second simulation, collision avoidance was set to Yes, so agents selected an unoccupied neighboring cell whenever one was available. This reduced the number of agents occupying the same cell at the same time. However, paths could still intersect because agents may visit the same location at different time steps. Increasing the number of agents from five to ten and the number of steps from 20 to 40 also produced denser paths and greater grid coverage. In all scenarios, the agents remained within the 10×10 grid boundaries.